

버튼·LCD 통합 실습

VS Code 사전 시뮬레이션 → 오픈소스 CLI → 보드 실험

같은 RTL에 10개 회로를 넣고 버튼으로 선택한다. LCD에 모드 번호와 회로 이름을 표시한다.

작성일 2026. 09. 10. · 이해리

1. 먼저 2,560개 입력과 버튼·LCD 동작을 검사한다.
2. 합성·배치배선·비트스트림을 생성한다.
3. 보드에서 모드를 순환하고 사진·영상으로 설명한다.

통합 실습 목차

1부 · VS Code 사전 시뮬레이션
clone·새 창·workspace·실행·파일형

2부 · 통합 회로 이해
10개 모드·버튼·LCD·보드 연결

3부 · 구현과 결과
CLI 설치·실행·검증 범위
보드·사진·영상·실험 후 레포트

전체 목차 PDF

시뮬레이션 도구 확인

Git·Python 3.10 이상·VS Code·Icarus Verilog를 설치한다. Windows PowerShell에서 한 줄씩 확인한다.

```
git --version  
python --version  
iverilog -V  
vvp -V
```

설치·PATH 변경 후 VS Code를 다시 연다. Linux·macOS·WSL은 python 대신 python3를 사용한다.

설치와 빈 템플릿 안내

v2.0.0 템플릿을 새 이름으로 clone

아래는 macOS·Linux·WSL의 Bash/Zsh 명령이다. 줄 끝 문자는 다음 줄로 명령을 이어 준다.

```
git clone --branch v2.0.0 \  
https://github.com/Glaysia/fpga-lab-template.git \  
lab1_integrated_cli \  
cd lab1_integrated_cli \  
git switch -c main \  
code LAB1.code-workspace
```

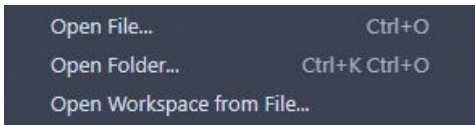
회로마다 마지막 폴더 이름을 바꾼다. 시작 파일은 주석뿐이며 코드 이미지를 보며 직접 작성한다.
고정된 수업 버전 v2.0.0

File → New Window

VS Code 상단 File → New Window. Ctrl+Shift+N도 같은 동작이다.



새 창의 File → Open Workspace from File...을 누른다.



프로젝트 하나의 workspace 열기

1. 방금 clone한 lab1_integrated_cli 폴더를 연다.
2. LAB1.code-workspace를 선택하고 Open. 모든 실습의 workspace 파일명은 같다.
3. Explorer에 PROJECT 하나와 src.sim.constraints.tools 폴더가 보이는지 확인한다.
4. src/design.v.sim/tb_design.v.constraints/pins.xdc는 아직 주석뿐이다.
5. simulation.json은 실행할 RTL·TB 파일과 TB top을 지정하는 설정이다.

추천 확장 설치

1. 왼쪽 Extensions 아이콘을 누른다.
2. 검색창에 @recommended를 입력한다.
3. slang의 Install: Verilog/SystemVerilog 편집·진단.
4. VaporView의 Install: VCD 파형 확인.
5. vscode-pdf의 Install: 코드 옆에서 PDF 교안 열기.

WSL 사용자는 WSL 창에서 필요할 경우 Install in WSL을 누른다. 확장 설치와 Python·Icarus 설치의 별개다.

src에 RTL 파일 만들기

1. Explorer에서 src 왼쪽 화살표를 눌러 펼친다.
 2. design.v 우클릭 → Rename → logic_gate.v 입력.
 3. 파일을 두 번 눌러 열고 뒤의 코드 이미지를 보며 전체 코드를 입력한다.
 4. 추가 RTL이 있으면 src 우클릭 → New File로 만든다.
 5. 전체 RTL 파일 목록은 다음 장의 src 표를 따른다.
 6. File → Save All로 모든 파일을 저장한다.
- 설계 모듈명: lab1_integrated. 파일명과 module 이름을 구분한다.

sim에 테스트벤치 만들기

1. sim을 펼치고 tb_design.v 우클릭 → Rename.
2. 새 파일명: tb_lab1_integrated.sv
3. 뒤의 TB 코드 이미지에 있는 선언·DUT 연결·입력 자극·예상값
검사를 직접 작성한다.
4. wave.vcd를 만드는 dumpfile·dumpvars와 종료 조건까지 작성한다.
5. TB 모듈명: tb_lab1_integrated
6. File → Save All. TB를 합성용 RTL 폴더에 넣지 않는다.

작성할 RTL 파일 목록 (1)

폴더	파일명
src	logic_gate.v
src	half_adder.v
src	full_adder.v
src	adder_4bit.v
src	sub_4bit.v
src	compare_4.v

각 파일을 New File로 만들고 코드 이미지의 전체 내용을 작성한다.
파일마다 module 선언과 endmodule을 확인한다.

작성할 RTL 파일 목록 (2)

폴더	파일명
src	mux_4x1.v
src	demux_1x8.v
src	encoder8x3.v
src	decoder3x8.v
src	seg_decoder.v
src	button_onepulse.v

각 파일을 New File로 만들고 코드 이미지의 전체 내용을 작성한다.
파일마다 module 선언과 endmodule을 확인한다.

작성할 RTL 파일 목록 (3)

폴더	파일명
src	lcd_modes.v
src	lab1_integrated.v

각 파일을 New File로 만들고 코드 이미지의 전체 내용을 작성한다.
파일마다 module 선언과 endmodule을 확인한다.

simulation.json을 내 파일에 맞추기

Explorer에서 simulation.json을 두 번 눌러 연다.

sources 배열: 앞의 모든 RTL 파일명 앞에 src/를 붙여 등록한다.

한 파일 예: "sources": ["src/logic_gate.v"]

testbench: sim/tb_lab1_integrated.sv

simulation_top: tb_lab1_integrated

sources의 각 경로와 testbench·simulation_top 값은 큰따옴표로 감싼다.
여러 경로는 쉼표로 구분한다.

파일명·확장자·괄호·쉼표를 확인하고 Save All.

통합에 포함할 개별 회로

01-10번에서 작성한 회로를 src에 모아 통합 top에 연결한다. 아래의 코드 화면도 그대로 재사용한다.

full_adder.v와 half_adder.v는 함께 필요하다. 통합 RTL·버튼·LCD까지 합성 파일은 총 14개다.

이미 작성한 파일은 복사해도 되며 simulation.json의 sources에 14개를 모두 등록한다.

개별 회로별 상세 설명으로 이동

개별 RTL 재사용 · logic_gate

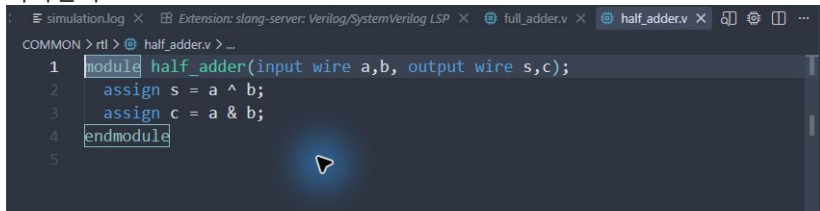
src/logic_gate.v · 전체 코드를 입력하거나 앞서 작성한 같은 파일을 복사한다.

```
1 module logic_gate(input wire a,b, output wire x,y,z);  
2     assign x = a & b;  
3     assign y = a | b;  
4     assign z = a ^ b;  
5 endmodule
```

모듈 이름과 입출력 폭을 유지하고 File → Save All.

개별 RTL 재사용 · half_adder

src/half_adder.v · 전체 코드를 입력하거나 앞서 작성한 같은 파일을 복사한다.

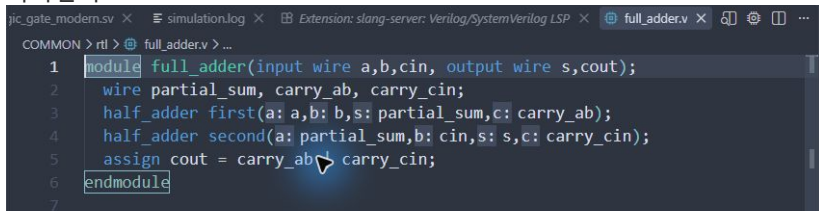


```
COMMON > rtl > half_adder.v > ...  
1 module half_adder(input wire a,b, output wire s,c);  
2     assign s = a ^ b;  
3     assign c = a & b;  
4 endmodule  
5
```

모듈 이름과 입출력 폭을 유지하고 File → Save All.

개별 RTL 재사용 · full_adder

src/full_adder.v · 전체 코드를 입력하거나 앞서 작성한 같은 파일을 복사한다.



```
COMMON > rtl > full_adder.v > ...
1 module full_adder(input wire a,b,cin, output wire s,cout);
2   wire partial_sum, carry_ab, carry_cin;
3   half_adder first(a: a,b: b,s: partial_sum,c: carry_ab);
4   half_adder second(a: partial_sum,b: cin,s: s,c: carry_cin);
5   assign cout = carry_ab & carry_cin;
6 endmodule
7
```

모듈 이름과 입출력 폭을 유지하고 File → Save All.

개별 RTL 재사용 · adder_4bit

src/adder_4bit.v · 전체 코드를 입력하거나 앞서 작성한 같은 파일을 복사한다.

```
COMMON > rtl > @ adder_4bit.v
1 module adder_4bit(input wire [3:0] a,b, output wire [3:0] s, output wire cout);
2   assign {cout,s} = {1'b0,a} + {1'b0,b};
3 endmodule
```

모듈 이름과 입출력 폭을 유지하고 File → Save All.

개별 RTL 재사용 · sub_4bit

src/sub_4bit.v · 전체 코드를 입력하거나 앞서 작성한 같은 파일을 복사한다.

```
COMMON > rtl > @ sub_4bit.v > ...  
1 module sub_4bit(input wire [3:0] a,b, output wire [3:0] d, output wire bor);  
2   assign d = a - b;  
3   assign bor = a < b; // Equal operands do NOT borrow.  
4 endmodule
```

모듈 이름과 입출력 폭을 유지하고 File → Save All.

개별 RTL 재사용 · compare_4

src/compare_4.v · 전체 코드를 입력하거나 앞서 작성한 같은 파일을 복사한다.

```
COMMON > rtl > @ compare_4.v > ...  
1 module compare_4(input wire [3:0] a,b, output wire [2:0] o);  
2   assign o = {a > b, a == b, a < b};  
3 endmodule
```

모듈 이름과 입출력 폭을 유지하고 File → Save All.

개별 RTL 재사용 · mux_4x1

src/mux_4x1.v · 전체 코드를 입력하거나 앞서 작성한 같은 파일을 복사한다.

```
1 module mux_4x1(input wire [3:0] i, input wire [1:0] s, output wire z);  
2     // Preserve the original lab ordering: 00 -> i[3].  
3     assign z = i[3-s];  
4 endmodule
```

모듈 이름과 입출력 폭을 유지하고 File → Save All.

개별 RTL 재사용 · demux_1x8

src/demux_1x8.v · 전체 코드를 입력하거나 앞서 작성한 같은 파일을 복사한다.

```
1 module demux_1x8(input wire i, input wire [2:0] s, output wire [7:0] o);  
2     // Preserve the original lab ordering: 000 -> o[7].  
3     assign o = i ? (8'b10000000 >> s) : 8'b0;  
4 endmodule
```

모듈 이름과 입출력 폭을 유지하고 File → Save All.

개별 RTL 재사용 · encoder8x3

src/encoder8x3.v · 전체 코드를 입력하거나 앞서 작성한 같은 파일을 복사한다.

```
1 module encoder8x3(input wire [7:0] i, output reg [2:0] a);
2     // One-hot only. Zero and multi-hot inputs return 000 (not a priority encoder).
3     always @* begin
4         case (i)
5             8'h80: a=0; 8'h40: a=1; 8'h20: a=2; 8'h10: a=3;
6             8'h08: a=4; 8'h04: a=5; 8'h02: a=6; 8'h01: a=7;
7             default: a=0;
8         endcase
9     end
10 endmodule
```

모듈 이름과 입출력 폭을 유지하고 File → Save All.

개별 RTL 재사용 · decoder3x8

src/decoder3x8.v · 전체 코드를 입력하거나 앞서 작성한 같은 파일을 복사한다.

```
1 module decoder3x8(input wire a,b,c, output wire [7:0] o);  
2   assign o = 8'b00000001 << {a,b,c};  
3 endmodule
```

모듈 이름과 입출력 폭을 유지하고 File → Save All.

개별 RTL 재사용 · seg_decoder

src/seg_decoder.v · 전체 코드를 입력하거나 앞서 작성한 같은 파일을 복사한다.

```
1 module seg_decoder(input wire [3:0] bcd, output reg [7:0] seg_data);
2   // [7:0] = {a,b,c,d,e,f,g,dp}; active high; hexadecimal 0..F; dp off.
3   always @* begin
4     case (bcd)
5       0:seg_data=8'hfc; 1:seg_data=8'h60; 2:seg_data=8'hda; 3:seg_data=8'hf2;
6       4:seg_data=8'h66; 5:seg_data=8'hb6; 6:seg_data=8'hbe; 7:seg_data=8'he0;
7       8:seg_data=8'hfe; 9:seg_data=8'hf6; 10:seg_data=8'hee; 11:seg_data=8'h3e;
8       12:seg_data=8'h9c; 13:seg_data=8'h7a; 14:seg_data=8'h9e; 15:seg_data=8'h8e;
9       default:seg_data=0;
10    endcase
11  end
12 endmodule
```

모듈 이름과 입출력 폭을 유지하고 File → Save All.

통합 RTL · 모드 번호

lab1_integrated.v · 1-11행 · 실제 VS Code 화면

```
1 // Input clock MUST be the trainer's 1 kHz main clock (B6).
2 // Mode button N8; reset K4; data DIPSW1..8 = sw[7]..sw[0].
3 module lab1_integrated #(parameter integer DEBOUNCE_CYCLES=20, LCD_POWER_WAIT=50)(
4     input wire clk,rst,mode_button, input wire [7:0] sw,
5     output reg [7:0] led, output wire [7:0] seg_data,
6     output wire lcd_e,lcd_rs,lcd_rw, output wire [7:0] lcd_data);
7     wire press; reg [3:0] mode;
8     button_onepulse #(DEBOUNCE_CYCLES) button(clk,rst,mode_button,press);
9     always @(posedge clk or posedge rst)
10         if(rst) mode<=0;
11         else if(press) mode<=(mode==9)?0:mode+1;
```

기본값은 디바운스 20클록·LCD 대기 50클록이다. 입력 클록은 보드의 1kHz다.

reset은 mode=0, 버튼 펄스는 다음 모드로 이동한다. 내부 번호 0-9는 LCD의 01-10에 대응한다.

통합 RTL · 논리 · 가산 연결

lab1_integrated.v · 12-18행 · 실제 VS Code 화면

```
12 wire gx,gy,gz,fs,fc,ac,borrow,mz;
13 wire [3:0] sum,difference;
14 wire [2:0] comparison,encoded;
15 wire [7:0] demuxed,decoded,hex_segments;
16 logic_gate gates(sw[7],sw[6],gx,gy,gz);
17 full_adder fa(sw[7],sw[6],sw[5],fs,fc);
18 adder_4bit add4(sw[7:4],sw[3:0],sum,ac);
```

공유 RTL의 논리 게이트·전가산기·4비트 가산기를 인스턴스화한다. DIP 입력에서 필요한 비트를 연결한다.

통합 RTL · 나머지 회로 연결

lab1_integrated.v · 19-25행 · 실제 VS Code 화면

```
19 sub_4bit sub4(sw[7:4],sw[3:0],difference,borrow);
20 compare_4 comp(sw[7:4],sw[3:0],comparison);
21 mux_4x1 mux(sw[7:4],sw[1:0],mz);
22 demux_1x8 demux(sw[7],sw[2:0],demuxed);
23 encoder8x3 encoder(sw,encoded);
24 decoder3x8 decoder(sw[2],sw[1],sw[0],decoded);
25 seg_decoder seven(sw[3:0],hex_segments);
```

감산·비교·MUX·DEMUX·인코더·디코더·7세그먼트도 같은 입력 스위치를 사용한다.

모든 회로는 함께 동작하며, 다음 코드에서 현재 모드의 출력을 선택한다.

통합 RTL · 출력 선택 01-05

lab1_integrated.v · 26-34행 · 실제 VS Code 화면

```
26 assign seg_data=(mode==9)?hex_segments:8'b0;  
27 always @* begin  
28     led=0;  
29     case(mode)  
30         0:led={gx,gy,gz,5'b0};  
31         1:led={fc,fs,6'b0};  
32         2:led={ac,sum,3'b0};  
33         3:led={borrow,difference,3'b0};  
34         4:led={comparison,5'b0};
```

7세그먼트는 mode=9에서만 켜다. LED 기본값을 0으로 두고 모드별 결과를 지정한다.

비트 묶음의 왼쪽이 led[7]이며 보드 LED1에 연결된다. 남은 비트는 0으로 채운다.

통합 RTL · 출력 선택 06-10

lab1_integrated.v · 35-44행 · 실제 VS Code 화면

```
35     5:led={mz,7'b0};
36     6:led=demuxed;
37     7:led={encoded,5'b0};
38     8:led=decoded;
39     9:led=hex_segments;
40     default:led=0;
41 endcase
42 end
43 lcd_modes #(LCD_POWER_WAIT) display(clk,rst,mode,lcd_e,lcd_rs,lcd_rw,lcd_data);
44 endmodule
```

MUX부터 7세그먼트까지 출력 선택을 마친다. LCD에는 같은 mode를 전달해 회로 이름을 표시한다.

버튼 RTL · 두 단계 동기화

button_onepulse.v · 1-9행 · 실제 VS Code 화면

```
1 module button_onepulse #(parameter integer STABLE_CYCLES=20)(
2     input wire clk,rst,button, output reg pulse);
3     (* ASYNC_REG="TRUE" *) reg sync1, sync2;
4     reg accepted;
5     integer count;
6     always @(posedge clk or posedge rst) begin
7         if(rst) begin sync1<=0; sync2<=0; end
8         else begin sync1<=button; sync2<=sync1; end
9     end
```

외부 버튼을 sync1, sync2 두 플립플롭을 거쳐 받는다. ASYNC_REG 속성으로 동기화 레지스터임을 표시한다.

버튼 RTL · 한 번만 전환

button_onepulse.v · 10-20행 · 실제 VS Code 화면

```
10  always @(posedge clk or posedge rst) begin
11      if(rst) begin count<=0; accepted<=0; pulse<=0; end
12      else begin
13          pulse<=0;
14          if(sync2==accepted) count<=0;
15          else if(count==STABLE_CYCLES-1) begin
16              accepted<=sync2; count<=0; pulse<=sync2;
17          end else count<=count+1;
18      end
19  end
20  endmodule
```

입력이 accepted와 다르게 20클럭 유지되면 새 상태를 받아들인다. 그 전에는 연속 안정 시간을 센다.

pulse의 기본값은 0이고 누름을 받아들일 때만 1이다. 땔 때는 상태만 바뀌어 모드를 넘기지 않는다.

LCD RTL · 포트와 상태

lcd_modes.v · 1-11행 · 실제 VS Code 화면

```
1 // Board source: 2022 CHARACTER LCD guide p13/p30, 1 kHz clock, 16x2, 8-bit bus.  
2 // Each byte has 1 ms setup, 1 ms enable-high and >=2 ms recovery.  
3 module lcd_modes #(parameter integer POWER_WAIT=50)(  
4     input wire clk,rst, input wire [3:0] mode,  
5     output reg lcd_e,lcd_rs, output wire lcd_rw, output reg [7:0] lcd_data);  
6     reg [1:0] phase;  
7     integer power_count;  
8     reg [5:0] index;  
9     reg [3:0] shown_mode;  
10    reg [127:0] name;  
11    assign lcd_rw=1'b0;
```

phase는 한 바이트의 전송 단계, index는 화면 내 위치다. shown_mode는 현재 화면의 모드다.

한 화면을 전송하는 동안 shown_mode를 유지하여 두 줄의 모드가 섞이지 않게 한다.

LCD RTL · 회로명 선택

lcd_modes.v · 12-21행 · 실제 VS Code 화면

```
12  always @* begin
13      case(shown_mode)
14          0:name="AND OR XOR      "; 1:name="FULL ADDER      ";
15          2:name="4 BIT ADDER      "; 3:name="4 BIT SUBTRACT ";
16          4:name="4 BIT COMPARE    "; 5:name="4 TO 1 MUX      ";
17          6:name="1 TO 8 DEMUX      "; 7:name="8 TO 3 ENCODER  ";
18          8:name="3 TO 8 DECODER    "; 9:name="HEX 7 SEGMENT ";
19          default:name="RESET REQUIRED ";
20      endcase
21  end
```

shown_mode에 따라 16문자 회로명을 고른다. 남는 자리는 공백으로 채운다.

한 화면을 전송하는 동안 shown_mode를 유지하여 두 줄의 모드가 섞이지 않게 한다.

LCD RTL · 리셋과 전원 대기

lcd_modes.v · 22-26행 · 실제 VS Code 화면

```
22  always @(posedge clk or posedge rst) begin
23      if(rst) begin
24          phase<=0; power_count<=0; index<=0; shown_mode<=0;
25          lcd_e<=0; lcd_rs<=0; lcd_data<=0;
26      end else if(power_count<POWER_WAIT) power_count<=power_count+1;
```

리셋 때 모든 상태와 출력 신호를 초기화한다. 1kHz에서 POWER_WAIT=50은 50ms 대기다.

한 화면을 전송하는 동안 shown_mode를 유지하여 두 줄의 모드가 섞이지 않게 한다.

LCD RTL · 초기 명령

lcd_modes.v · 27-34행 · 실제 VS Code 화면

```
27  else case(phase)
28      0:begin
29          lcd_e<=0; lcd_rs<=0;
30          case(index)
31              0,1,2:lcd_data<=8'h38;
32              3:lcd_data<=8'h0c;
33              4:lcd_data<=8'h06;
34              5:begin lcd_data<=8'h80; shown_mode<=mode; end
```

38을 세 번 보낸 뒤 0C, 06으로 초기화한다. 첫 줄 주소 80을 보낼 때 mode를 shown_mode에 저장한다.

한 화면을 전송하는 동안 shown_mode를 유지하여 두 줄의 모드가 섞이지 않게 한다.

LCD RTL · MODE 01-10

lcd_modes.v · 35-42행 · 실제 VS Code 화면

```
35: 6:begin lcd_rs<=1; lcd_data<="M"; end
36: 7:begin lcd_rs<=1; lcd_data<="O"; end
37: 8:begin lcd_rs<=1; lcd_data<="D"; end
38: 9:begin lcd_rs<=1; lcd_data<="E"; end
39: 10:begin lcd_rs<=1; lcd_data<=" "; end
40: 11:begin lcd_rs<=1; lcd_data<=(shown_mode==9)?"1":"0"; end
41: 12:begin lcd_rs<=1; lcd_data<=(shown_mode==9)?"0":("1"+shown_mode); end
42: 22:lcd_data<=8'hc0;
```

문자 MODE와 두 자리 번호를 보낸다. 내부 mode=9만 10으로 표시하고, 그다음 C0으로 둘째 줄을 선택한다.

한 화면을 전송하는 동안 shown_mode를 유지하여 두 줄의 모드가 섞이지 않게 한다.

LCD RTL · 이름의 한 글자

lcd_modes.v · 43-49행 · 실제 VS Code 화면

```
43  default:begin
44      lcd_rs<=1;
45      if(index>=23 && index<=38) lcd_data<=name[127-(index-23)*8 -: 8];
46      else lcd_data<=" ";
47      end
48  endcase
49  phase<=1;
```

index=23-38에서 128비트 이름을 8비트씩 꺼낸다. 나머지 첫 줄 위치는 공백으로 채운다.

한 화면을 전송하는 동안 shown_mode를 유지하여 두 줄의 모드가 섞이지 않게 한다.

LCD RTL · E 펄스와 반복

lcd_modes.v · 50-60행 · 실제 VS Code 화면

```
50     end
51     1:begin lcd_e<=1; phase<=2; end
52     2:begin lcd_e<=0; phase<=3; end
53     3:begin
54         // First function-set recovery is 6 ms including setup of the next byte.
55         if(index==0 && power_count<POWER_WAIT+2) power_count<=power_count+1;
56         else begin phase<=0; index<=(index==38)?5:index+1; end
57     end
58 endcase
59 end
60 endmodule
```

phase 1에서 E를 올리고 phase 2에서 내린다. 1kHz이므로 E high는 1ms다.

첫 초기 명령 뒤에는 추가 대기한다. 한 화면이 끝나면 index=5로 돌아가 모드를 새로 저장한다.

통합 TB · 검사 환경

tb_lab1_integrated.sv · 1-13행 · 실제 VS Code 화면

```
1 | timescale 1ns/1ps
2 | module tb_lab1_integrated;
3 |   reg clk=0,rst=1,mode_button=0; reg [7:0] sw=0;
4 |   wire [7:0] led,seg_data,lcd_data; wire lcd_e,lcd_rs,lcd_rw;
5 |   lab1_integrated #(.DEBOUNCE_CYCLES(4),.LCD_POWER_WAIT(5)) dut(.*);
6 |   always #5 clk=~clk;
7 |   integer checked=0,m,n,k,v,frames=0,addr=0;
8 |   reg [7:0] expected,expected_seg;
9 |   reg [127:0] line1,line2,expected_name,completed_line1,completed_line2;
10 |  reg [55:0] expected_heading;
11 |  integer init_count=0;
12 |  reg [7:0] init_words[0:4];
13 |  reg [7:0] digits[0:15];
```

10ns 클럭과 짧은 디바운스·전원 대기로 기능 검사를 빠르게 한다. 실제 합성의 1kHz 설정과 구분한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 반복 동작

tb_lab1_integrated.sv · 14~22행 · 실제 VS Code 화면

```
14 task cycles(input integer amount);  
15     repeat(amount) @(negedge clk);  
16 endtask  
17 task assert_mode(input integer value);  
18     if(dut.mode!=value) $fatal(1,"Mode expected %0d actual %0d",value,dut.mode);  
19 endtask  
20 task press_once;  
21     mode_button=1; cycles(12); mode_button=0; cycles(12);  
22 endtask
```

cycles는 클록 하강까지 기다리고 assert_mode는 모드 번호를 검사한다.
press_once는 눌렀다 떼는 한 번의 동작이다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · LCD 명령 검사

tb_lab1_integrated.sv · 23-32행 · 실제 VS Code 화면

```
23 // Decode the actual external LCD bus at E falling edge.
24 always @(negedge lcd_e) if(!rst) begin
25     if(lcd_rw!=0) $fatal(1,"LCD must write only");
26     if(!lcd_rs) begin
27         if(init_count<5) begin
28             if(lcd_data!=init_words[init_count]) $fatal(1,"LCD initialization command %0d got %h",init_count,
29                 lcd_data);
30             init_count=init_count+1;
31         end
32         if(lcd_data==8'h80) begin addr=0;line1=0;end
33         else if(lcd_data==8'hc0) begin addr=64;line2=0;end
```

실제 출력 lcd_e의 하강에서 lcd_rw와 초기 명령을 검사한다. 주소 80·C0을 만나면 해당 줄 조립을 시작한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 완성된 화면

tb_lab1_integrated.sv · 33-41행 · 실제 VS Code 화면

```
33   end else begin
34       if(addr<16) line1[127-addr*8 -: 8]=lcd_data;
35       if(addr>=64 && addr<80) line2[127-(addr-64)*8 -: 8]=lcd_data;
36       if(addr==79) begin
37           completed_line1=line1;completed_line2=line2;frames=frames+1;
38       end
39       addr=addr+1;
40   end
41 end
```

주소에 따라 받은 문자를 16문자 버퍼에 넣는다. 둘째 줄 마지막 문자까지 받은 순간 완성된 두 줄을 보관한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 리셋과 바운스

tb_lab1_integrated.sv · 42-52행 · 실제 VS Code 화면

```
42  initial begin
43      init_words[0]=8'h38;init_words[1]=8'h38;init_words[2]=8'h38;init_words[3]=8'h0c;init_words[4]=8'h06;
44      digits[0]=252;digits[1]=96;digits[2]=218;digits[3]=242;
45      digits[4]=102;digits[5]=182;digits[6]=190;digits[7]=224;
46      digits[8]=254;digits[9]=246;digits[10]=238;digits[11]=62;
47      digits[12]=156;digits[13]=122;digits[14]=158;digits[15]=142;
48      $dumpfile("wave.vcd");$dumpvars(0,tb_lab1_integrated);
49      cycles(3);rst=0;cycles(12);assert_mode(0);
50      // Bounces shorter than four samples must not be accepted.
51      repeat(3) begin mode_button=1;cycles(1);mode_button=0;cycles(2);end
52      cycles(12);assert_mode(0);
```

기대 LCD 명령·숫자 패턴을 준비한다. 1클록 누름과 2클록 땜을 반복해도 mode=0인지 확인한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 모드별 입력

tb_lab1_integrated.sv · 53-60행 · 실제 VS Code 화면

```
53  for(m=0;m<10;m=m+1) begin
54      assert_mode(m);
55  for(n=0;n<256;n=n+1) begin
56      sw=n;cycles(1);expected=0;expected_seg=0;
57      case(m)
58      0:expected={sw[7]&sw[6],sw[7]|sw[6],sw[7]^sw[6],5'b0};
59      1:expected=(int'(sw[7])+int'(sw[6])+int'(sw[5]))<<6;
60      2:expected=((n/16)+(n%16))<<3;
```

10개 모드 각각에서 DIP 256개를 모두 넣는다. 출력의 비트 배치까지 포함하여 별도 기대값을 계산한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 경계와 선택 순서

tb_lab1_integrated.sv · 61-68행 · 실제 VS Code 화면

```
61      3:expected=(((n/16)<(n%16))?16:0)|(((n/16)-(n%16))&15))<<3;
62      4:expected=((n/16)>(n%16))?4:(((n/16)==(n%16))?2:1)<<5;
63      5:expected=(((n/16)>>(3-(n%4)))&1)<<7;
64      6:expected=(n>=128)?(128>>(n%8)):0;
65      7:begin for(k=0;k<8;k=k+1)if(n==(128>>k))expected=k<<5;end
66      8:expected=1<<(n%8);
67      9:begin expected=digits[n%16];expected_seg=expected;end
68      endcase
```

감산 borrow, 비교 결과, 선택순 순서, 인코더의 비정상 입력까지 기대값에 반영한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 출력과 모드 번호

tb_lab1_integrated.sv · 69-79행 · 실제 VS Code 화면

```
69     if(led!=expected || seg_data!=expected_seg)
70     | $fatal(1,"mode=%0d sw=%h expected LED=%h got=%h",m,sw,expected,led);
71     checked=checked+1;
72     end
73     // Allow two complete 16x2 refreshes, including a mode change midway through a frame.
74     cycles(350);
75     expected_heading="MODE 00";
76     expected_heading[15:8]=(m==9)?8'h31:8'h30;
77     expected_heading[7:0]=(m==9)?8'h30:(8'h31+m);
78     if(completed_line1[127:72]!=expected_heading || init_count!=5)
79     | $fatal(1,"LCD mode text mismatch: %s",completed_line1);
```

LED·7세그먼트를 매 입력마다 비교한다. 화면 갱신을 기다린 뒤 LCD 첫 줄의 MODE 번호와 초기화 횟수를 검사한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 회로명 대조

tb_lab1_integrated.sv · 80-87행 · 실제 VS Code 화면

```
80  case(m)
81      0:expected_name="AND OR XOR      ";1:expected_name="FULL ADDER      ";
82      2:expected_name="4 BIT ADDER     ";3:expected_name="4 BIT SUBTRACT  ";
83      4:expected_name="4 BIT COMPARE    ";5:expected_name="4 TO 1 MUX      ";
84      6:expected_name="1 TO 8 DEMUX     ";7:expected_name="8 TO 3 ENCODER  ";
85      8:expected_name="3 TO 8 DECODER   ";9:expected_name="HEX 7 SEGMENT  ";
86  endcase
87  if(completed_line2!=expected_name)$fatal(1,"LCD circuit text mismatch: %s",completed_line2);
```

두 번째 줄은 16문자 전체를 해당 모드의 기대 회로명과 비교한다.
번호만 맞고 이름이 틀린 오류도 잡는다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께
설명한다.

통합 TB · 길게 누름 · 순환 · 복귀

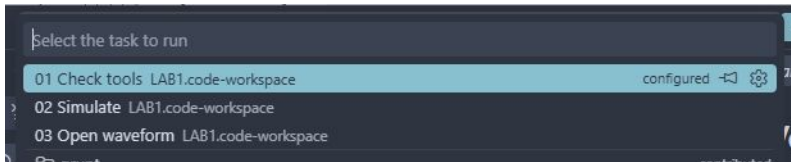
tb_lab1_integrated.sv · 88-99행 · 실제 VS Code 화면

```
88 // A long press advances once and only once.
89 mode_button=1;cycles(80);assert_mode((m==9)?0:m+1);
90 mode_button=0;cycles(12);
91 end
92 assert_mode(0);press_once;assert_mode(1);
93 init_count=0;rst=1;cycles(3);rst=0;cycles(200);assert_mode(0);
94 if(init_count!=5 || completed_line1[127:72]!="MODE 01")$fatal(1,"LCD reset recovery failed");
95 if(checkered!=2560 || frames<10)$fatal(1,"Incomplete integrated test");
96 $display("LAB1_PASS lab1_integrated cases=%0d",checked);$finish;
97 end
98 initial begin #1000000;$fatal(1,"Watchdog");end
99 endmodule
```

80클럭 동안 눌러도 한 번만 넘어가는지, 10→01 순환과 재리셋 후 LCD 복구를 검사한다.

2,560개 출력과 10개 이상 화면을 확인한 뒤 PASS를 출력한다. 외부 보드의 실제 동작은 별도 실험이다.

세 작업을 순서대로 실행



1. Terminal → Run Task... → 01 Check tools.
2. 02 Simulate → 종료까지 기다린다.
3. 터미널과 build/sim/run-.../simulation.log를 확인한다.
4. LAB1_PASS lab1_integrated cases=2560을 확인한다.
5. 03 Open waveform으로 wave.vcd를 연다.

모든 workspace의 사전 시뮬레이션은 Icarus Verilog다. SIMULATED와 TB의 실제 검사 결과를 구분한다.

통합 테스트의 검사 범위

10개 모드 × DIP 256개 = 2,560개 출력 비교.

reset → MODE 01, 짧은 바운스 무시, 길게 눌러도 한 번만 전환, 10→01 순환을 검사한다.

LCD 초기 명령과 외부 버스에서 완성된 두 줄의 번호·이름을 비교한다.

TB는 10ns 클록, 디바운스 4클록, 전원 대기 5클록으로 시간을 줄인다.
실제 합성은 1kHz 보드 설정이다.

Icarus 검사 종료: 72430ns. 실제 보드의 전압·LCD 연결은 별도로 확인한다.

VaporView에서 통합 파형 읽기

1. wave.vcd가 텍스트면 탭 우클릭 → Reopen Editor With... → VaporView.
2. Netlist에서 tb_lab1_integrated를 펼친다.
3. clk, rst, mode_button, sw, led, seg_data를 추가한다.
4. dut 안의 mode와 lcd_e, lcd_rs, lcd_rw, lcd_data를 추가한다.
5. Zoom to Fit 뒤 모드가 바뀌는 구간을 확대한다.
6. 버튼 입력부터 모드 변경까지의 지연과 LCD 한 화면 갱신을 구분한다.

constraints에 XDC 직접 작성

1. constraints를 펼치고 pins.xdc 우클릭 → Rename.
2. 파일명: lab1_integrated.xdc
3. 핀 표와 코드 이미지를 보며 PACKAGE_PIN·IOSTANDARD·get_ports를 입력한다.
4. get_ports의 이름과 비트 번호를 자신이 작성한 RTL의 포트와 대조한다.
5. File → Save All. Vivado 또는 CLI 구현에는 이 파일을 직접 가져간다.

XDC는 Icarus 기능 시뮬레이션에서 사용하지 않는다. 파형 통과만으로 핀 배치까지 검증됐다고 판단하지 않는다.

XDC 전체 코드 · 1-13행

constraints/lab1_integrated.xdc · 실제 VS Code 화면을 보며 직접 작성한다.

```
1  # Combo II-DLD Spartan-7: 2022 LCD guide p13 (1 kHz), p30 and S7 pin map.
2  set_property PACKAGE_PIN B6 [get_ports {clk}]
3  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
4  set_property PACKAGE_PIN K4 [get_ports {rst}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
6  set_property PACKAGE_PIN N8 [get_ports {mode_button}]
7  set_property IOSTANDARD LVCMOS33 [get_ports {mode_button}]
8  set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
9  set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
10 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
12 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
13 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
```

핀 이름·포트 이름·대괄호·중괄호를 확인하고 저장한다.

XDC 전체 코드 · 14-26행

constraints/lab1_integrated.xdc · 실제 VS Code 화면을 보며 직접 작성한다.

```
14 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
16 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
18 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
19 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
20 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
21 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
22 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
24 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
25 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
26 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
```

핀 이름·포트 이름·대괄호·중괄호를 확인하고 저장한다.

XDC 전체 코드 · 27-38행

constraints/lab1_integrated.xdc · 실제 VS Code 화면을 보며 직접 작성한다.

```
27 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
28 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
30 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
31 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
32 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
33 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
34 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
36 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
37 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
38 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
```

핀 이름·포트 이름·대괄호·중괄호를 확인하고 저장한다.

XDC 전체 코드 · 39-52행

constraints/lab1_integrated.xdc · 실제 VS Code 화면을 보며 직접 작성한다.

```
39 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
40 set_property PACKAGE_PIN R6 [get_ports {seg_data[0]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[0]}]
42 set_property PACKAGE_PIN R4 [get_ports {seg_data[1]}]
43 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[1]}]
44 set_property PACKAGE_PIN R2 [get_ports {seg_data[2]}]
45 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[2]}]
46 set_property PACKAGE_PIN T5 [get_ports {seg_data[3]}]
47 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[3]}]
48 set_property PACKAGE_PIN N3 [get_ports {seg_data[4]}]
49 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[4]}]
50 set_property PACKAGE_PIN P7 [get_ports {seg_data[5]}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[5]}]
52 set_property PACKAGE_PIN P3 [get_ports {seg_data[6]}]
```

핀 이름·포트 이름·대괄호·중괄호를 확인하고 저장한다.

XDC 전체 코드 · 53-65행

constraints/lab1_integrated.xdc · 실제 VS Code 화면을 보며 직접 작성한다.

```
53 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[6]]]
54 set_property PACKAGE_PIN P1 [get_ports {seg_data[7]]]
55 set_property IOSTANDARD LVCMOS33 [get_ports {seg_data[7]]]
56 set_property PACKAGE_PIN A4 [get_ports {lcd_data[0]]]
57 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[0]]]
58 set_property PACKAGE_PIN B2 [get_ports {lcd_data[1]]]
59 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[1]]]
60 set_property PACKAGE_PIN C3 [get_ports {lcd_data[2]]]
61 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[2]]]
62 set_property PACKAGE_PIN D4 [get_ports {lcd_data[3]]]
63 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[3]]]
64 set_property PACKAGE_PIN A2 [get_ports {lcd_data[4]]]
65 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[4]]]
```

핀 이름·포트 이름·대괄호·중괄호를 확인하고 저장한다.

XDC 전체 코드 · 66~78행

constraints/lab1_integrated.xdc · 실제 VS Code 화면을 보며 직접 작성한다.

```
66 set_property PACKAGE_PIN C5 [get_ports {lcd_data[5]]}
67 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[5]]}
68 set_property PACKAGE_PIN C1 [get_ports {lcd_data[6]]}
69 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[6]]}
70 set_property PACKAGE_PIN D1 [get_ports {lcd_data[7]]}
71 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data[7]]}
72 set_property PACKAGE_PIN A6 [get_ports {lcd_e}]
73 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_e}]
74 set_property PACKAGE_PIN G6 [get_ports {lcd_rs}]
75 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_rs}]
76 set_property PACKAGE_PIN D6 [get_ports {lcd_rw}]
77 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_rw}]
78 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
```

핀 이름·포트 이름·대괄호·중괄호를 확인하고 저장한다.

코드 수정 실험 · 실패와 복구

1. 정상 PASS 로그를 보관한다. Explorer → src → lab1_integrated.v을 연다.
2. 버튼 처리의 모드 순환 비교값을 9에서 8로 바꾼다. 테스트벤치의 기대값은 그대로 둔다.
3. File → Save All → Terminal → Run Task... → 02 Simulate.
4. 내부 mode는 0부터 센다. 화면 MODE 09 다음에 MODE 10 대신 MODE 01로 돌아가면 실패다.
5. 이번 실행의 실패 로그에서 입력·기대값·실제값을 읽는다. 실행 폴더를 기록한다.
6. 바꾼 부분을 원래대로 복구하고 Save All → 02 Simulate. 전체 PASS 와 새 파형을 확인한다.

사전 레포트에 변경 이유·실패 원인·복구 결과를 비교한다. 복구 PASS 뒤에 구현으로 진행한다.

제작 검증: 정상 → 실패 검출 → 복구



실험 전 레포트에 넣을 것

1. 모드별 예상 입력·출력과 비트 순서를 표로 작성한다.
2. 10개 회로·버튼·LCD 파일의 역할을 설명한다.
3. PASS 로그, VCD, 모드 전환 및 LCD 파형 캡처를 연결한다.
4. TB에서 줄인 시간과 실제 1kHz 설정을 구분한다.
5. 실험에서 확인할 버튼·LCD·DIP·LED 장면을 계획한다.

레포트 양식과 예시

통합 구조와 입력 배치

lab1_integrated → 10개 조합회로 + button_onepulse + lcd_modes.

clk=B6(1kHz), KEY1=reset, KEY2=mode_button.

DIP1-8은 sw[7:0], LED1-8은 led[7:0]에 대응한다.

개별 실습의 KEY 입력은 통합본에서 DIP로 옮겨 모드 버튼과 충돌하지 않게 한다.

단일 7세그먼트는 모드 10에서만 동작한다.

통합 동작 명세와 핀표

모드별 조작 1-5

모드	입력	출력
01	a=DIP1, b=DIP2	LED1=AND, 2=OR, 3=XOR
02	a,b,cin=DIP1,2,3	LED1=carry, 2=sum
03	a=DIP1-4, b=DIP5-8	LED1=carry, 2-5=sum
04	a=DIP1-4, b=DIP5-8	LED1=borrow, 2-5=diff
05	a=DIP1-4, b=DIP5-8	LED1=콤, 2=같음, 3=작음

사용하지 않는 LED 비트는 0이다. reset하면 모드 01.

모드별 조작 6-10

모드	입력	출력
06	i=DIP1-4, s=DIP7-8	LED1=i[3-s]
07	i=DIP1, s=DIP6-8	s=0→LED1, s=7→LED8
08	DIP1-8 중 하나만 1	LED1-3: DIP1→0, DIP8→7
09	abc=DIP6,7,8	0→LED8, 7→LED1
10	hex=DIP5-8	a-dp 패턴, 1이면 점등

사용하지 않는 LED 비트는 0이다. reset하면 모드 01.

버튼을 길게 누르면

2단 동기화 → 안정된 입력 20클록 확인 → 상승 시 1클록 펄스.

1kHz에서 약 20ms 안정 시간에 동기화 지연이 추가된다.

누른 채 유지하면 한 번만 바뀐다. 떼면 뒤 다시 눌러야 다음 모드가 된다.

모드 10 다음은 01. reset은 즉시 01로 돌아간다.

실험 영상에서 짧게 누르기·길게 누르기·순환을 각각 보여준다.

LCD가 표시되는 과정

전원 대기 50ms → 38,38,38,0C,06 초기 명령 → 두 줄 반복 갱신.

첫 줄 MODE 01...MODE 10, 두 번째 줄 회로 이름.

E high는 1ms, 바이트 간격은 최소 4ms. 첫 초기 명령 간격은 더 길다.

한 화면 시작 때 모드를 저장하여 두 줄이 다른 모드로 섞이지 않게 한다.

새 모드는 현재 전송을 마친 뒤 갱신되므로 잠깐 기다리고 읽는다.

CLI 실행 환경

macOS ARM64 공식 바이너리 배포를 확인하고 WSL Ubuntu-26.04에서 Linux x86-64 바이너리로 실행했다.

APIO 1.5.1, OSS CAD Suite 2026.08.19, openXC7 2026.08.20.

Yosys → nextpnr → frames → bit까지 정확한 S75로 성공했다.

Mac 직접 실행과 실제 보드 프로그래밍은 이 제작 검증에 포함하지 않는다.

공식 ARM64·Linux 배포

도구 설치

macOS·WSL의 Bash/Zsh 터미널에서 실행한다.

```
python3 -m venv ~/.local/share/fpga-apio
source ~/.local/share/fpga-apio/bin/activate
pip install apio==1.5.1
apio packages install
```

WSL에 venv가 없으면 먼저 python3-venv를 설치한다.

macOS는 darwin-arm64, WSL은 linux-x86-64 배포를 사용한다.
설치·실행 명령 전체

도구 경로 확인

```
export PATH="$HOME/.apio/packages/oss-cad-suite/bin:$PATH"  
export PATH="$HOME/.apio/packages/openxc7/bin:$PATH"  
yosys --version  
nextpnr-xilinx --version
```

새 터미널에는 PATH 설정을 다시 적용한다.

LD_LIBRARY_PATH로 두 묶음의 lib를 섞지 않는다. 배포 래퍼가 자신의 라이브러리를 선택한다.

정확한 S75 데이터 준비

대상 부품은 xc7s75fgga484-1이다.

upstream PR 15가 S75를 올바른 S100 fabric에 연결한다. S75와 같은 실리콘 구조를 공유하는 매핑이며 부품명·IDCODE는 S75다.

해당 fgga484 패키지의 실제 338개 I/O 핀을 적용한다. 학생은 Vivado를 설치·실행할 필요가 없다.

준비 도구는 공개 DB 커밋과 핀 표 해시를 고정하고 모두 검사한다.
데이터 수정의 근거

도구 보조 파일 받기

아래는 회로 프로젝트와 별개인 도구 데이터 폴더다.

```
mkdir -p ~/fpga-cli/lab1-tools
cd ~/fpga-cli/lab1-tools
repo=fpga-lab-example-opensource-cli-integrated
base=https://raw.githubusercontent.com/Claysia/$repo/v2.0.1/tools
curl --fail --location "$base/prepare_s75.py" -o prepare_s75.py
pins=xc7s75fgga484-1-package-pins.csv
curl --fail --location "$base/$pins" -o "$pins"
```

고정된 보조 도구 태그는 v2.0.1, 학생 빈 템플릿 태그는 v2.0.0이다.

S75 chipdb 생성

```
python3 prepare_s75.py
```

다운로드·bbaexport·bbasm 종료를 기다린다.

~/fpga-cli/lab1-s75/prepared.json에서 part와 338개 핀, DB 커밋·chipdb 해시를 확인한다.

실패하면 같은 폴더의 export.log·bbasm.log를 읽는다. 메모리 부족일 때 다른 무거운 앱을 닫고 다시 실행한다.

이후 VS Code의 학생 프로젝트 터미널로 돌아간다.

내 프로젝트에서 사전 검사

clone한 workspace에서 RTL 14개·TB·XDC·simulation.json을 직접 작성한다.

프로젝트 루트는 src·sim·constraints가 있는 폴더다.

```
python3 tools/lab1.py simulate  
mkdir -p build/cli
```

2,560개 입력 검사와 버튼·LCD 검사를 확인한다.

실험 전 레포트의 예상값·로그·파형을 저장한 뒤 합성으로 이동한다.

합성 명령 파일을 직접 작성

Explorer의 PROJECT 우클릭 → New File → synth.js. 다음 네 줄을 작성하고 저장한다.

```
read_verilog src/*.v
synth_xilinx -family xc7 -top lab1_integrated
write_json build/cli/design.json
stat
```

src에는 합성할 RTL만 두고 TB는 sim에 둔다.

Yosys 실행과 결과 읽기

```
yosys -l build/cli/synthesis.log -s synth.js
```

오류 없이 끝나는지와 build/cli/design.json의 갱신을 확인한다.

stat의 셀 종류·개수를 읽고 top이 lab1_integrated인지 확인한다.

합성 로그를 evidence/cli에 보관한다. 실패했으면 원인을 수정하고 이 명령부터 다시 실행한다.

nextpnr 배치배선

```
chip="$HOME/fpga-cli/lab1-s75/xc7s75fgga484-1.bin"  
db="$HOME/fpga-cli/lab1-s75/prjxray-db/spartan7"  
part=xc7s75fgga484-1  
nextpnr-xilinx --chipdb "$chip" \  
--xdc constraints/lab1_integrated.xdc \  
--json build/cli/design.json --fasm build/cli/design.fasm \  
--freq 0.001 --log build/cli/place-route.log
```

B6의 1kHz를 MHz로 표현하면 0.001이다. XDC와 같은 클록이다.

K4/B6 핀 오류가 나면 이전 S50 기반 chipdb를 사용했는지 확인한다.

프레임 변환

```
fasm2frames --db-root "$db" --part "$part" \  
build/cli/design.fasm build/cli/design.frames
```

앞 장과 같은 터미널을 사용한다. db와 part 변수를 유지한다.

배치배선이 성공한 새 design.fasm을 사용한다.

종료 코드 0과 design.frames의 갱신을 확인한다.

해당 S75 DB와 다른 부품의 part.yaml을 섞지 않는다.

bit 파일 생성

```
xc7frames2bit --part_file "$db/$part/part.yaml" \  
--part_name "$part" --frm_file build/cli/design.frames \  
--output_file build/cli/lab1_integrated.bit
```

build/cli/lab1_integrated.bit가 이번 실행으로 생성됐는지 확인한다.
실행 명령·종료 코드·파일 크기·생성 시각을 실험 후 레포트에 보관한다.
제작 검증에서는 3,686,926바이트가 생성됐다.

생성한 bit 다시 읽기

```
bitread --part_file "$db/$part/part.yaml" -C -z -y \  
-o build/cli/bits.txt build/cli/lab1_integrated.bit
```

체크섬 검사와 Number of configuration frames: 9104, DONE을 확인한다.

이 검사는 파일 구조를 확인한다. 실제 장치 기록·LCD·버튼 동작은 다음 실험 단계에서 확인한다.

제작 실행의 9,104프레임 읽기 로그

검증 결과와 다음 실험

Icarus 통합 자기검사 2,560개 통과.

Yosys·nextpnr·fasm2frames·xc7frames2bit 종료 코드 모두 0.

정확한 S75 bit 생성 및 9,104프레임 체크섬 검사 통과.

소스·XDC·DB·chipdb 해시와 실행 로그를 공개했다.

다음은 openFPGALoader로 실제 장치에 기록하고 모드·LCD·입출력을
사진과 영상으로 검증한다.

전체 명령·근거·기록 절차

대상 부품·핀 제약 (1)

xc7s75fpga484-1 · IOSTANDARD=LVCNMOS33

포트	PACKAGE_PIN
clk	B6
rst	K4
mode_button	N8
sw[0]	U4
sw[1]	V4
sw[2]	W1

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (2)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
sw[3]	W4
sw[4]	T1
sw[5]	U2
sw[6]	W3
sw[7]	Y1
led[0]	N5

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (3)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
led[1]	M1
led[2]	M3
led[3]	M7
led[4]	N7
led[5]	M2
led[6]	M4

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (4)

xc7s75fgga484-1 · IOSTANDARD=LVCMOS33

포트	PACKAGE_PIN
led[7]	L4
seg_data[0]	R6
seg_data[1]	R4
seg_data[2]	R2
seg_data[3]	T5
seg_data[4]	N3

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (5)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
seg_data[5]	P7
seg_data[6]	P3
seg_data[7]	P1
lcd_data[0]	A4
lcd_data[1]	B2
lcd_data[2]	C3

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (6)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
lcd_data[3]	D4
lcd_data[4]	A2
lcd_data[5]	C5
lcd_data[6]	C1
lcd_data[7]	D1
lcd_e	A6

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (7)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
lcd_rs	G6
lcd_rw	D6

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

openFPGALoader 설치

bit 생성과 FPGA 기록은 별도 단계다. macOS는 아래 명령을 사용한다.

```
brew install openfpgaloader
```

Windows MSYS2 UCRT64 환경에서는 다음 명령을 사용한다.

```
pacman -S mingw-w64-ucrt-x86_64-openFPGALoader
```

Linux는 배포판 패키지 또는 공식 빌드 안내를 따른다. 설치한 환경에서 확인한다.

```
openFPGALoader --help
```

공식 설치·기록 안내

보드와 JTAG 케이블 확인

1. 보드 전원과 USB JTAG 케이블을 연결한다.
2. openFPGALoader -list-cables로 지원 케이블 식별자를 확인한다.
3. openFPGALoader -list-boards로 지원 보드 식별자를 확인한다.
4. 보드가 목록에 없으면 실제 연결된 JTAG 케이블의 식별자를 -c에 지정한다.
5. VM 또는 WSL에서는 해당 실행 환경에 USB 장치가 연결되어 있는지 확인한다.

다른 FPGA 보드 이름을 임의로 선택하지 않는다. 보드 모델과 케이블 모델은 별개의 정보다.

CLI로 생성한 bit를 SRAM에 기록

아래 MY_CABLE은 직전 단계에서 확인한 실제 케이블 식별자로 바꾼다.

```
openFPGALoader -c MY_CABLE build/cli/lab1_integrated.bit
```

자신의 소스로 만든 최신 bit인지 경로·생성 로그·해시를 확인한 뒤 실행한다.

터미널의 장치 정보·진행률·종료 코드와 최종 상태를 보관한다.

기본 기록은 SRAM이며 전원을 끄면 사라진다. -f는 Flash 기록 옵션이므로 이 SRAM 실습 명령에는 넣지 않는다.

SRAM·Flash와 케이블 옵션

기록 뒤 모드·LCD 동작 확인

KEY1=reset, KEY2=다음 모드. DIP1-8=sw[7:0]. LED1-8=led[7:0].

reset으로 01을 확인하고 버튼을 짧게·길게 눌러 한 번씩 전환되는지 본다.

10 다음 01로 돌아오는지, LCD 번호·이름이 실제 선택 회로와 일치하는지 확인한다.

모드마다 DIP를 조작하여 예상표의 정상·경계 조건과 LED·7세그먼트 출력을 비교한다.

기록 성공 로그와 실제 회로 동작을 각각 증빙한다.

CLI 실험 후 레포트와 GitHub

1. Icarus·Yosys·nextpnr·프레임 변환·openFPGALoader 버전과 실행 명령을 기록한다.
2. RTL·TB·XDC 커밋, 정확한 part, DB 커밋과 bit 해시를 연결한다.
3. evidence/cli에 합성·배치배선·bit 생성 로그를 보관한다.
4. evidence/programming에 케이블·장치·기록 로그와 캡처를 보관한다.
5. 보드 사진과 10개 모드 시연 영상에 조건·타임스탬프를 붙인다.
6. 사전 파형·예상값·실측 차이를 해석하고 reports/post에서 증빙을 링크한다.

전체 목차 PDF